UNIVERSIDAD TÉCNICA DE MANABÍ

FACULTAD DE CIENCIAS INFORMÁTICAS

CARRERA DE TECNOLOGIAS DE LA INFORMACION

ASIGNATURA

DESARROLLO DE SISTEMAS INFORMÁTICOS

ACTIVIDAD 1:

ESTUDIO DE CASO: SISTEMA DE CONTROL DE RESCATE Y ADOPCIÓN ANIMAL

NIVEL:

QUINTO SEMESTRE

ESTUDIANTE:

GUEVARA CAICHE ANGIE DALILA

DOCENTE:

ING. JORGE ZAMBRANO CEDEÑO, MG

PERIODO:

SEPTIEMBRE 2026 – ENERO 2027

1 ANÁLISIS DEL PROBLEMA

Diseñar un sistema para la gestión de casos en un refugio animal que permita reportar ingresos de animales rescatados (perros, gatos, fauna silvestre), asignar voluntarios o veterinarios encargados y gestionar estados del proceso (ej. Rescatado, En Tratamiento, Adopción). Para lograr una arquitectura clara y profesional, se deberá utilizar obligatoriamente un repositorio en GitHub bajo el modelo Gitflow y aplicar al menos tres patrones de diseño de software, justificando su elección y uso técnico.

Tabla 1. Actores del sistema y sus responsabilidades.

Actor	**Responsabilidad dentro del sistema.**
Ciudadano	Reportar un animal que necesita rescate y proporcionar información disponible.
Voluntario	Atender el caso asignado, registrar y actualizar la información de su intervención.
Veterinario	Evaluar al animal, registrar diagnósticos y tratamientos y actualizar su seguimiento.
Administrador	Gestionar casos, asignar personal, supervisar procesos y cambios de estado.

Los actores identificados interactúan de forma complementaria: el ciudadano inicia el flujo al reportar un caso; el voluntario y el veterinario ejecutan las acciones operativas y clínicas; y el administrador supervisa y controla el ciclo completo. Esta separación de responsabilidades permite aplicar el principio de mantenibilidad y facilita la asignación de permisos según el rol.

Definir el alcance:

El alcance del sistema va desde el registro inicial de un caso de rescate hasta la resolución del mismo (adopción o liberación). No se incluyen en esta versión módulos de gestión financiera, inventario de alimentos ni integración con redes sociales, los cuales podrían incorporarse en fases posteriores.

- Registrar un caso de rescate.
- Clasificar al animal como perro, gato o fauna silvestre.
- Consultar los casos registrados.
- Asignar un voluntario o veterinario.
- Actualizar el estado del caso.
- Registrar observaciones o tratamientos.
- Consultar el historial de cambios.
- Emitir una notificación cuando se registra un nuevo rescate.

Reglas de negocio:

- Cada caso debe tener un identificador único. (Modelado)
- No se puede asignar un responsable que no esté registrado. (Flujo)

- Cada cambio de estado debe quedar registrado. (Restricción)
- Los datos clínicos deben ser gestionados por personal autorizado. (Restricción)
- Un caso no debería cerrarse sin registrar el resultado de la intervención. (Flujo)

Ciclo de vida del caso de rescate:

Se definen los estados por los que atraviesa un animal desde su ingreso hasta su resolución:

Rescatado → En evaluación → En tratamiento → En adopción → Adoptado

Cada transición debe quedar registrada y solo puede ser ejecutada por personal autorizado, garantizando así la trazabilidad exigida en las reglas de negocio.

2 REQUERIMIENTOS DEL SISTEMA

Requerimientos funcionales:

Los requerimientos funcionales se derivaron directamente del alcance y de las reglas de negocio definidas en la sección anterior, asegurando que cada funcionalidad del sistema responda a una necesidad concreta de los actores identificados.

- Registrar un nuevo caso de rescate.
- Clasificar el animal por especie.
- Consultar los casos registrados.
- Asignar un voluntario o veterinario.
- Actualizar el estado de un caso.
- Registrar observaciones y datos de seguimiento.
- Notificar al personal correspondiente sobre nuevos rescates.
- Consultar el historial de un caso.
- Registrar la resolución del caso.

Requerimientos no funcionales:

Tabla 2. Requerimientos no funcionales y su aplicación en el proyecto

Requerimiento	**Aplicación en el proyecto**
Escalabilidad	Permitir incorporar más casos, usuarios o especies sin rediseñar todo en sistema.
Mantenibilidad	Separar las responsabilidades para que los cambios no afecten a otros módulos.
Trazabilidad	Registrar las modificaciones del código mediante Git.
Seguridad	Restringir operaciones según el rol del usuario.
Fiabilidad	Validar datos y evitar asignaciones o transacciones de estado inválidas.

3 CREACIÓN DEL REPOSITORIO

Ilustración 1. Registro de los datos de identificación del usuario en Git Bash

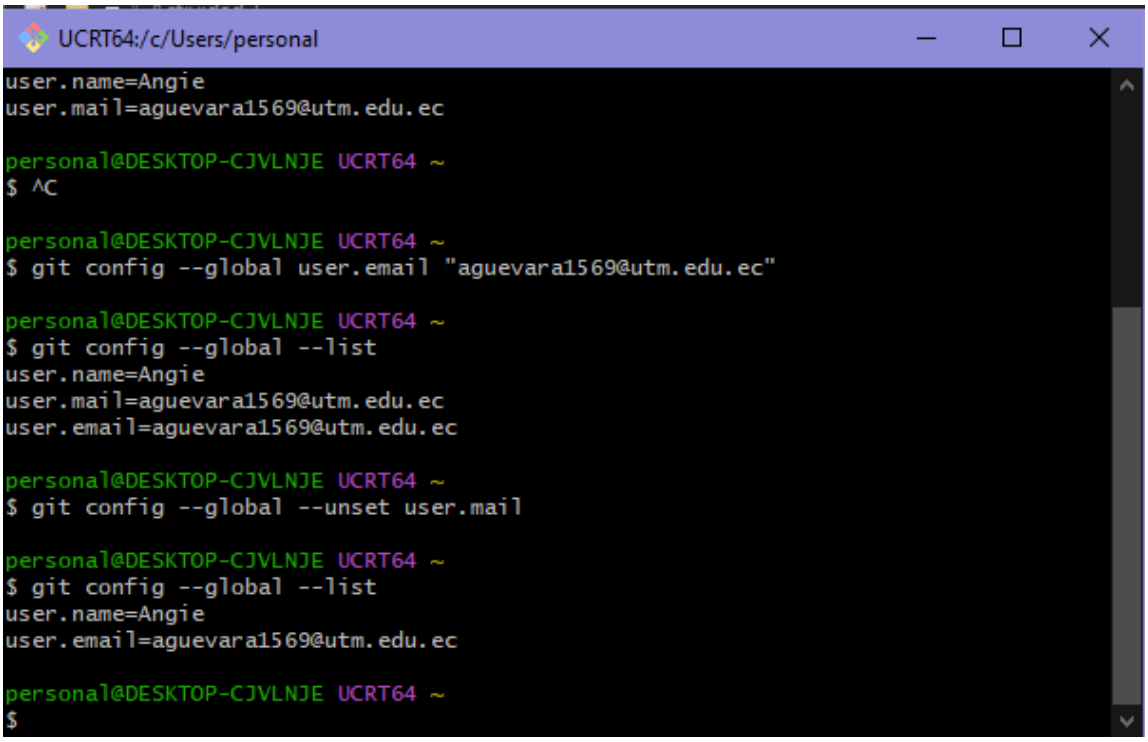
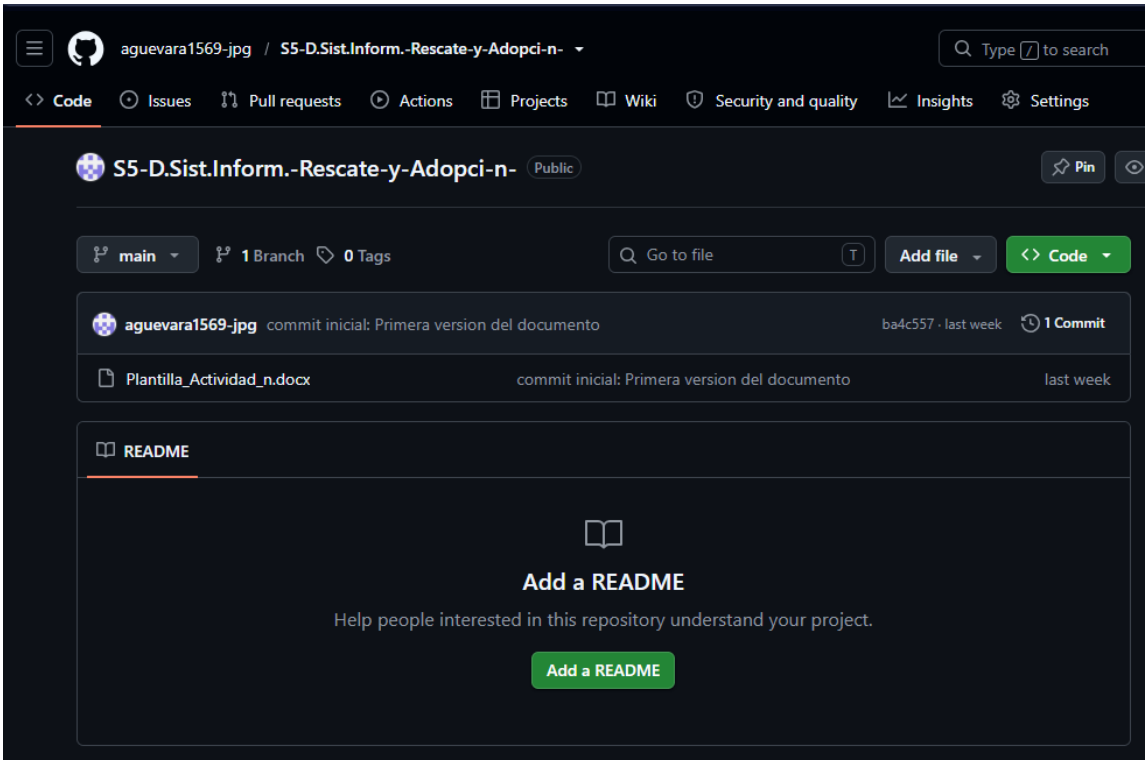


Ilustración 2. Repositorio del proyecto en GitHub



4 DISEÑO DE LA ARQUITECTURA DEL SISTEMA

Diagrama de casos de uso:

El diagrama de casos de uso ilustra el alcance funcional del sistema para el Centro de Rescate, representando las interacciones entre los actores principales y las capacidades clave de la plataforma. En él se modela cómo el Ciudadano/Reportante inicia el flujo mediante el reporte de rescates; el Administrador gestiona el ciclo operativo (registro de casos, clasificación de especies, asignación de voluntarios y consulta general); el Veterinario atiende la parte clínica realizando el registro de tratamientos y la revisión de historiales; y el Voluntario colabora en el seguimiento actualizando el estado de los casos asignados y consultando la información del sistema.

Ilustración 3. Diagrama de casos de uso del sistema

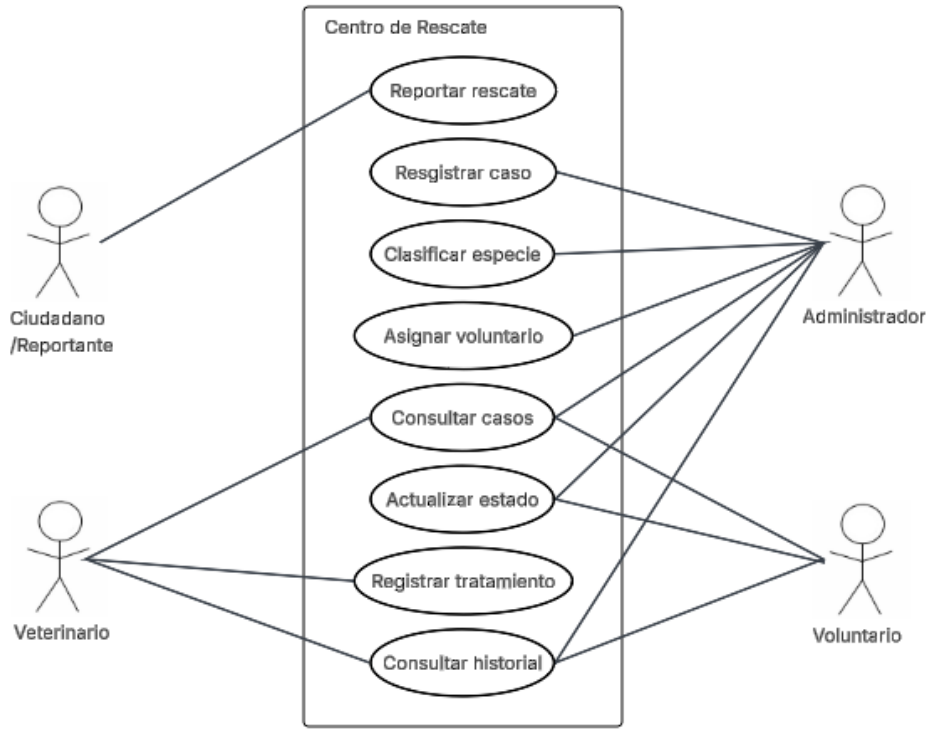
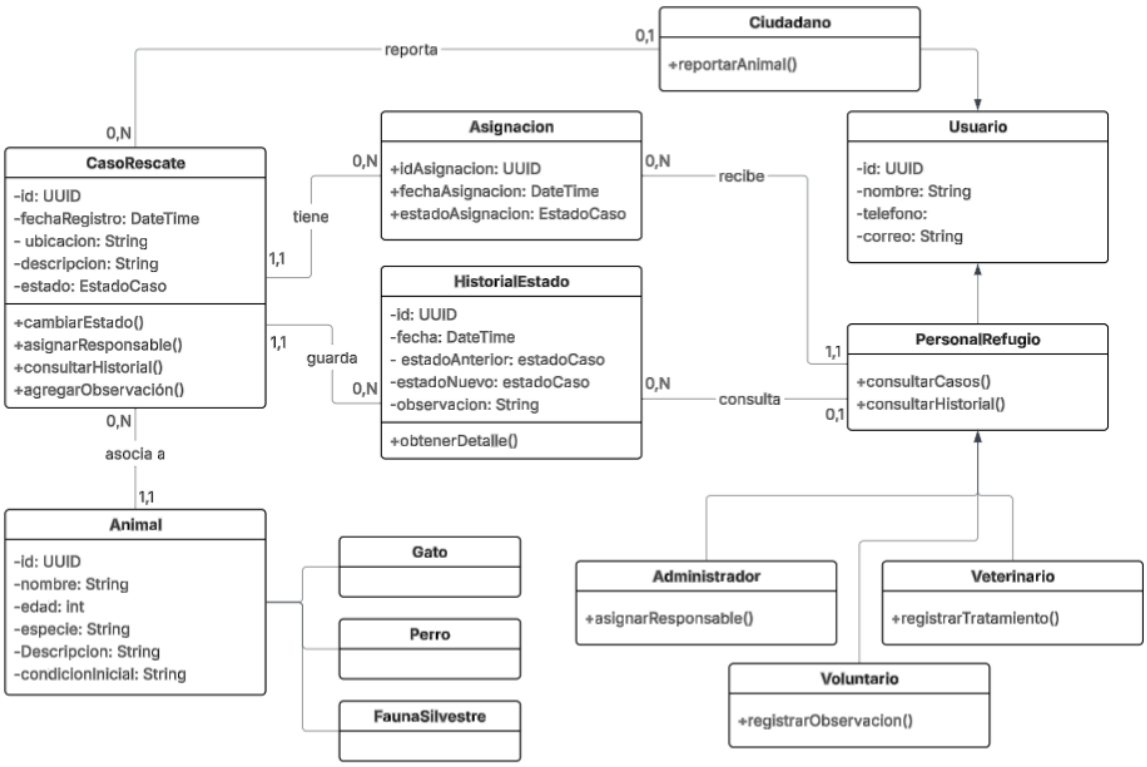


Diagrama de clases:

El diagrama modela la arquitectura estática del sistema para el centro de rescate mediante un conjunto de clases y sus relaciones. En él destaca una jerarquía donde la clase Usuario agrupa los datos comunes de los actores internos como PersonalRefugio (subdividido en Administrador, Veterinario y Voluntario) y el Ciudadano. Por otro lado, el núcleo operativo se compone de las clases CasoRescate y Animal. Esta última con especializaciones para tipos de especies. Complementadas por entidades de gestión y trazabilidad como Asignacion e HistorialEstado.

Ilustración 4. Diagrama de clases del dominio



5 APLICACIÓN DE PATRONES DE DISEÑO

Factory Method: Utilizado para incorporar otros tipos de animales mediante nuevos creadores, sin concentrar toda la creación en una sola clase.

- Clases: CreadorAnimal, CreadorPerro, CreadorGato, CreadorFaunaSilvestre.
- CreadorAnimal declara el método crearAnimal(). Cada creador concreto lo redefine para producir un tipo de animal.

Por ejemplo, CreadorGato crea un objeto Gato, que luego se asocia con un CasoRescate.

Observer: Permite añadir otros destinatarios o mecanismos de notificación sin tener que modificar toda la lógica de gestión de casos.

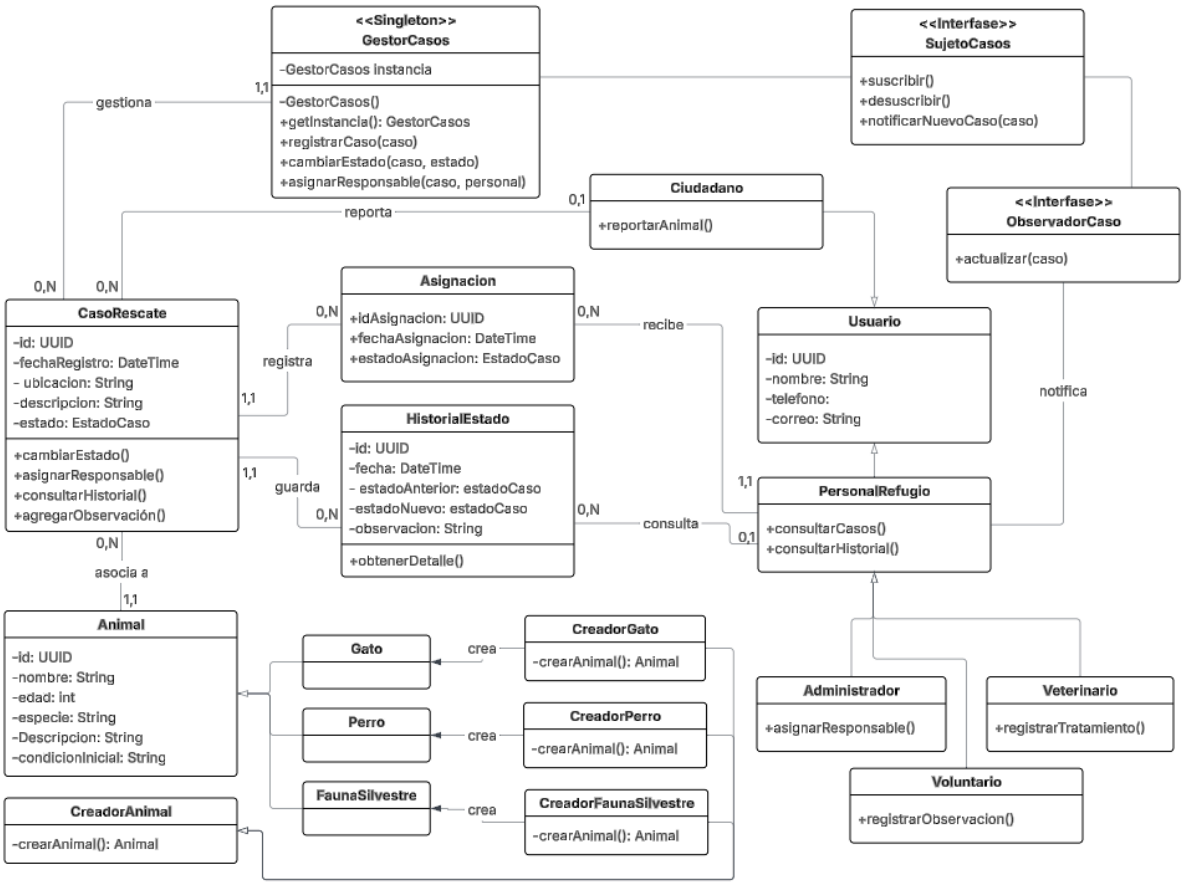
- Clases: SujetoCasos, ObservadorCaso, GestorCasos y PersonalRefugio.
- GestorCasos actúa como sujeto: mantiene suscripciones y notifica cuando se registra un nuevo caso. PersonalRefugio implementa la interfaz de observador para recibir la notificación.

Singleton: Permite centralizar el acceso al gestor en el ámbito definido para la aplicación. En este proyecto se limita al gestor central y no a los casos individuales.

- Clase: GestorCasos.

- El constructor privado -GestorCasos() impide crear instancias desde fuera de la clase, mientras que getInstancia() permite acceder a la instancia única controlada.

Ilustración 5. Aplicación de patrones de diseño Factory Method, Observer y Singleton en el sistema



6 REFERENCIAS

Larman, C. (2003). UML y patrones: Una introducción al análisis y diseño orientado a objetos y al proceso unificado (2.ª ed.). Pearson Educación.

Penadés Gramage, M. C. (2016). Diagrama de clases. Universitat Politècnica de València. https://riunet.upv.es/handle/10251/67392

Un Programador Nace. (2023, 27 de julio). Patrones de diseño: Singleton [Video]. YouTube. https://www.youtube.com/watch?v=cNoxcc0ca4I

Universitat Politècnica de València. (2022, 24 de febrero). Casos de uso y diagramas de casos de uso [Video]. YouTube. https://www.youtube.com/watch?v=iFcDoP6jEeE